Chaos GameDay Lab — Demo-3 Three-Tier Edition

Student Exercise — DEFCON Coimbatore 2026

> *"Break your own systems before the attacker does. Then break them again, in four different places."*

Duration: 2 hours | **Teams:** 2–6 people | **Skill Level:** Intermediate (Docker + basic SQL)

Why Demo-3 GameDay Is Harder Than Demo-2

Demo-2's GameDay taught one pattern: kill a stateless nginx, watch it respawn. Demo-3 forces teams to handle **four fundamentally different failure modes** in the same session:

Failure mode	Where state lives	Recovery primitive
Stateless wipeout	Nowhere (golden image)	Respawn
State loss (sessions)	Process memory	Respawn + accept loss
State preservation (transactions)	PostgreSQL WAL	Respawn + DB reconnect
Data-in-flight corruption	Primary + replica	Fence replica, PITR, resume
Real request blocking	ModSecurity CRS rules	Block in-flight, never reach app

A team that wins this GameDay has demonstrated they can defend across all four.

Setup (20 minutes)

```
# Stack prereqs
cd /Users/kuttanadan/Documents/defcon-demo/Demo-3
./setup_and_run3.sh setup     # First time only (~10-15 min)
./setup_and_run3.sh start     # Brings up all 5 containers + guardian + agent

# Open dashboard — this is the audience screen
open http://localhost:7880

# Confirm all healthy
./setup_and_run3.sh check
# Expected: 5 × (healthy) on the docker ps line
```

Team Roles

Role	Responsibility	Artefact
Attacker	Runs the card's `attack3_*.sh` — or crafts a custom variant	Command log
Defender	Watches `guardian3.sh` + dashboard, calls out state transitions	`/tmp/demo3_guardian.jsonl` excerpt
Timekeeper	Stopwatch — records V (time from command issued to service dark)	MTTR table
Analyst	Fills scorecard, flags surprises, logs gaps	GameDay scorecard

---

Attack Cards

Six cards covering all four tiers. Teams draw a card, run the attack, and are graded on whether the defender's R beats the card's target V.

🔴 CARD 1 — Web Tier Wipeout (Easy)

```
TIER:       WEB (demo3-web)
MITRE:      T1485 — Data Destruction
TARGET V:   5 seconds
TARGET R:   < 2 seconds
SCRIPT:     ./setup_and_run3.sh attack web

WHAT IT DOES:
  docker exec demo3-web sh -c 'rm -rf /usr/share/nginx/html/*'
  → HTTP returns 404 → guardian sees drift → respawn from golden image

VERIFY IMPACT:
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost:9091
  # Before respawn: 404
  # After respawn:  200

VERIFY RECOVERY:
  grep RESPAWN_SUCCESS /tmp/demo3_guardian.jsonl | tail -1

SCORING:
  Attacker +10  if service 404s for > 2s
  Defender +100 - (R_ms / 10) points for recovery
  Bonus    +20  if defender narrates the "golden image" phrase correctly
```

🟠 CARD 2 — App Tier Cascade (Medium)

```
TIER:       APP (demo3-app + DB for comparison)
MITRE:      T1499 — Endpoint DoS + T1529 — System Shutdown/Reboot
TARGET V:   5 seconds
TARGET R:   < 1 second
SCRIPT:     ./setup_and_run3.sh attack app

WHAT IT DOES:
  1. Create 3 in-memory sessions (victim_1, victim_2, victim_3)
  2. Inject 50 MB memory balloon via python3 bytearray
  3. docker kill demo3-app
  → Sessions vanish (in-memory). Transactions survive (Postgres).

VERIFY STATE LOSS:
  # Before attack:
  curl -s localhost:3001/status | python3 -m json.tool
  # → sessions_active: 3, transactions: 503

  # After respawn:
  curl -s localhost:3001/status | python3 -m json.tool
  # → sessions_active: 0 (LOST),  transactions: 503 (PRESERVED)

TEACHING POINT:
  Stateless = respawnable. Stateful = needs DB. This is the point.

SCORING:
  Attacker +15  if service health fails for > 2s
  Defender +100 - (R_ms / 10) for recovery
  Bonus    +25  if defender correctly explains which state was lost and which wa
```

🔴 CARD 3 — DB Ransomware (Hard)

```
TIER:       DB (demo3-db-pri + demo3-db-rep)
MITRE:      T1486 — Data Encrypted for Impact
TARGET V:   30 seconds (demo-adjusted — real T1486 can be minutes)
TARGET R:   < 15 seconds (fence-and-resume window)
SCRIPT:     ./setup_and_run3.sh attack db

WHAT IT DOES:
  1. Seed 500 rows (attack surface)
  2. 5 waves of 200-row mass UPDATE (amount *= -1)
  3. Write velocity spikes to 700-900 rows/sec
  → Guardian sees delta > 500 → MASS_WRITE_DETECTED → pg_wal_replay_pause
  → 10 quiet ticks (~10s) later → pg_wal_replay_resume

VERIFY DETECTION:
  grep MASS_WRITE_DETECTED /tmp/demo3_guardian.jsonl | tail -1
  # → rows_per_sec=700+

VERIFY FENCE:
  # During the fence window (~10s):
  docker exec demo3-db-rep psql -U phoenix -c "SELECT pg_is_wal_replay_paused();
  # → t (true)

VERIFY RESUME:
  grep REPLICA_RESUMED /tmp/demo3_guardian.jsonl | tail -1
  # → mttr_ms populated (~12500)

HONEST GAP (acknowledge in debrief):
  ~500 rows still reach the replica during the bulk INSERT phase before
  the velocity threshold computes. Demo shows *detection + containment*,
  not perfect bleed-zero. Real defence needs sub-100ms CDC monitoring.

SCORING:
  Attacker +30  if fence does not engage within 3s of first UPDATE wave
  Defender +100 - ((R_ms - 10000) / 100) for recovery within window
  Bonus    +40  if defender correctly describes why the bulk INSERT slips throug
```

🟠 CARD 4 — WAF Gauntlet (Medium)

```
TIER:       WAF (demo3-waf — ModSecurity + OWASP CRS)
MITRE:      T1190 + T1059.007 + T1083 + T1548 + T1595.002
TARGET V:   The attacker does NOT want V — they want a successful exploit
TARGET R:   0 — blocks happen in-flight, no recovery required
SCRIPT:     ./setup_and_run3.sh attack waf

WHAT IT DOES:
  15 real curl requests covering:
    - Wave 1: Reconnaissance (sqlmap UA, Nikto UA)
    - Wave 2: SQLi (OR 1=1, UNION SELECT, SLEEP(5), DROP TABLE)
    - Wave 3: XSS (<script>, <img onerror>, <svg onload>)
    - Wave 4: Path traversal (../etc/passwd, ../app/.env, null-byte)
    - Wave 5: Bypass (X-Forwarded-For, X-HTTP-Method-Override)

VERIFY BLOCKS:
  grep REQUEST_BLOCKED /tmp/demo3_guardian.jsonl | wc -l
  # Expected: 13

  docker exec demo3-waf tail -5 /var/log/modsec/audit.log
  # Real ModSecurity audit entries

HONEST GAP:
  1/15 passes (HTTP 404 — route didn't exist). Zero successful exploits.
  Block rate 13/15 = 86%. Don't claim 100% — the audience will notice.

SCORING:
  Defender +15 per real 403 (max 195)
  Defender +50 bonus if they correctly explain why the 404 isn't a miss
  Attacker -10 per wave that achieved 0 successful exploits
```

🟣 CARD 5 — Cascade (Expert)

```
TIERS:      ALL FOUR — simultaneously
TARGET V:   60 seconds
TARGET R:   C (combined cascade) ≤ 30 seconds

WHAT IT DOES:
  # Run in separate terminals, within 5s of each other:
  ./attack3_web.sh &
  ./attack3_app.sh &
  ./attack3_db.sh  &
  ./attack3_waf.sh &
  wait

VERIFY:
  curl -s http://localhost:7880/state | \
    python3 -c "import sys,json; d=json.load(sys.stdin); print('C =', d['rollup'

SCORING:
  +200 if dashboard shows all four tiers flip FAILED → RESTORED within 30s
  +100 bonus if the combined (C) cascade MTTR < 15s
  -50 per tier that remains FAILED at T+30s
```

🟣 CARD 6 — The Supply Chain Question (Thinking Card)

```
TIERS:      Conceptual — does not run code

PROMPT:
  The golden image pattern has one fatal assumption: the image itself is clean.
  What happens if the Dockerfile is modified? Or a base image pulls in a
  backdoored dependency? Every respawn ships poisoned.

  Describe — in ≤5 minutes at the whiteboard — how you would:
  1. DETECT a poisoned golden image before it ships
  2. PREVENT the poison from being built in the first place
  3. CONTAIN the blast radius if it shipped anyway

EVALUATION RUBRIC:
  +50 per mechanism named (cosign, SBOM/Syft, SLSA provenance, admission
      controllers, registry scanning, CI signing)
  +30 if the team mentions *both* build-time and runtime signals
  +20 if they tie it back to the Level 4 CTF supply-chain challenge
```

Scoring Rubric

Attacker Points

Achievement	Points
WEB wipeout — dashboard turns red	+10
APP kill — sessions lost verified	+15
DB ransomware — fence takes > 3s to engage	+30
WAF — any request returns 200	+50 (bonus — very hard)
Cascade — all four tiers failed simultaneously	+75
Found a guardian gap not in this doc	+100

Defender Points

MTTR (measured, not self-reported)	Points
WEB $R_1 < 1s$	+100
APP $R_2 < 1s$	+100
DB $R_3 < 15s$	+100
WAF block rate ≥ 80%	+100
Combined C < 20s	+150
Narrates the state-loss vs DB-preserved distinction clearly	+50
Acknowledges honest gaps openly in debrief	+75

WIN CONDITION: Team with highest total defender score.

GameDay Scorecard

Round	Card	Attacker	Defender	R (ms)	V (s)	R < V?	Notes
1							
2							
3							
4							
5							
6							
TOTAL							

Debrief Guide (30 minutes)

1. Measured vs Estimated

- What was your average R across all attacks?
- What was the **slowest** tier? Why?
- Was R < V in every round? If not, which card did you lose?

2. The State Question

- Which cards recovered without losing state?
- Which cards recovered *by* losing state (and was that the right call)?
- In production, which of your services is "stateless like WEB" vs "stateful like APP" vs "data-replicated like DB"?

3. The Honest Gaps

- Card 3 has a deliberate gap (bulk INSERT leak). Did your defender call it out?
- Card 4 has an honesty requirement (86%, not 100%). Did your defender maintain it?
- What other gaps did you find that aren't in this doc?

4. Real-World Extrapolation

- If WEB respawn is `docker run`, what's the Kubernetes equivalent? (`kubectl rollout restart`)
- If DB fence is `pg_wal_replay_pause`, what's the managed-DB equivalent? (RDS point-in-time recovery + paused replication)
- If guardian is bash, what's the production equivalent? (Prometheus rules + Alertmanager + operator reconcile loops)

5. The Next Step

- Pick one service at your employer. What is its R today?
- What would it take to get R under V for that service?
- Who do you need to convince? (CIO/CTO, not the security team.)

Bonus Challenge — Tighten The DB Fence

Card 3 has an honest gap: ~500 rows reach the replica before the fence engages. The 1-second poll is the bottleneck.

Challenge: Propose a change to `guardian3.sh` that closes this gap. Score:

- +30 if the proposal is architecturally sound (logical replication slot watcher, `pg_logical_emit_message`, row-count delta from a trigger)

- +50 if the team actually prototypes it
- +100 if their prototype fences the replica with < 50 row leak on a 5-wave attack

Teams that ship a working prototype go home with the unofficial "Bleed-Zero" award.

Instructor Notes

Common Student Questions

Q: Why isn't R_3 < 1s like R_1 and R_2? A: DB recovery isn't about killing and respawning a process. It's about deciding whether the replica's current LSN is a clean recovery point. That decision costs time — the 10 quiet ticks are the minimum viable "storm is over" signal. In production with sub-millisecond CDC monitoring, R_3 could drop to ~1s. But without real-time write-stream analysis, 10s is the floor.

Q: Why 86% WAF block rate and not 100%? A: Because the one that passed returned HTTP 404 — the route didn't exist. The request was never dangerous. Calling that a "miss" would be dishonest. The whole point of this talk is resisting vanity metrics, including one's own.

Q: Can I really `docker exec` into a production container? A: Not directly — but any RCE, SSRF with container-local effect, deserialisation, or compromised CI pipeline achieves the same result. `docker exec` in the demo is a stand-in for "attacker has arbitrary code exec inside the container."

Q: What's the Kubernetes version of this whole demo? A: Each tier becomes a Deployment with an immutable image, a liveness probe, and PodDisruptionBudget=0. Guardian becomes an operator's reconcile loop. The DB fence becomes a CRD that pauses a logical replication slot. The WAF becomes an Envoy sidecar with the ModSecurity filter. Same primitives, more YAML.

DEFCON Coimbatore 2026 · Demo-3 · Venugopal Parameswara